

Documentación técnica Smart Seller POS (v1.0 DNPA)

Propósito: Información consolidada para juntas directivas, auditoría o traspaso de conocimiento.

Alcance: Proyecto SmartSeller_POS_v1.0_DNPA (código fuente actual en repositorio).

Versión aplicación (pubspec): 1.0.0+1

Fecha de elaboración del documento: marzo 2026

1. Identificación del producto

Concepto	Detalle
-----	-----
Nombre comercial / interno	Smart Seller
Paquete Flutter	smart_seller
Descripción declarada	Sistema POS Smart Seller
Variante de código	DNPA línea de producto con formularios y flujos adaptados (incl. facturación electrónica y campos DIAN en productos)
Repositorio (referencia habitual)	GitHub ingeniero82/POS, rama feature/facturacion-electronica-completa
Idioma de interfaz	Español (localización es, es_CO)

2. Arquitectura y tecnología

2.1 Stack principal

Capa	Tecnología
-----	-----
Framework UI	Flutter (Google)
Lenguaje	Dart SDK >=3.0.0 <4.0.0
Diseño UI	Material Design 3 (useMaterial3: true)
Navegación y estado global	GetX (get: ^4.6.6) rutas nombradas, inyección de servicios (Get.put), middleware de autenticación
Base de datos local	SQLite vía sqflite + en escritorio sqflite_common_ffi y sqflite3_flutter_libs
Persistencia ligera	shared_preferences (p. ej. permisos por rol, licencia)
Archivos y rutas	path, path_provider

2.2 Plataforma de despliegue actual (operación típica)

- Windows (escritorio) es el entorno principal de ejecución documentado en el proyecto (flutter run -d windows, Visual Studio para toolchain).
- El mismo código Flutter puede compilarse para otras plataformas soportadas por Flutter si se configuran los runners correspondientes; la impresión serie y SQLite FFI están orientadas al uso en desktop.

2.3 Dependencias relevantes (negocio / integración)

Paquete	Uso
-----	-----
intl	Formato de moneda, fechas
pdf	Generación de PDF (reportes)

Paquete	Uso
excel, csv	Exportación de datos
file_picker, image_picker	Importación de archivos e imágenes de producto
cached_network_image	Caché de imágenes
flutter_typeahead	Autocompletado en formularios
libserialport	Comunicación con impresoras térmicas por puerto serie
crypto	Operaciones criptográficas (p. ej. hashing en seguridad)

2.4 Estructura lógica del código (lib/)

- main.dart: arranque, inicialización de BD, servicios globales (Auth, Permisos, Impresora, Config empresa), definición de todas las rutas GetX.
- screens/: pantallas principales del POS, inventario, usuarios, reportes clásicos, login, dashboard, etc.
- services/: acceso a datos SQLite, impresión, reportes, licencia, autorización, etc.
- models/: entidades (usuario, producto, venta, cliente, permisos, etc.).
- widgets/: componentes reutilizables (formulario de producto, menú de reimpresión, modales).
- modules/accounting/: contabilidad / reportes contables y cuentas por cobrar/pagar.
- modules/electronic_invoicing/: facturación electrónica (DIAN): pantallas, servicios de documentos, cola de pendientes, estado, validaciones, respaldo.
- middleware/: control de acceso a rutas (invitado vs autenticado).

3. Base de datos local (SQLite)

3.1 Versión de esquema

- Versión actual del esquema: 6 (SQLiteDatabaseService, migraciones en _onUpgrade).
- Los datos viven en el equipo (no es un SaaS multi-tenant en servidor propio del código analizado).

3.2 Tablas principales (resumen funcional)

Lista no exhaustiva de extensiones por migraciones; las núcleo incluyen:

Área	Tablas / conceptos
-----	-----
Seguridad	users (usuario, rol, contraseña, código de usuario opcional, activo)
Catálogo	groups (grupos/categorías visuales), products (código, precio, costo, stock, IVA 0/5/19%, pesables, imagen, etc.)
Inventario	inventory_movements (movimientos con tipo, cantidad, motivo, usuario)
Ventas	sales (fecha, total, usuario, método de pago, ítems en JSON, descuentos, devoluciones isReturn, anulación anulada, anulada_at, anulada_por)
Cientes POS	customers (puntos, compras, datos de contacto)
Empresa	company_config (razón social, NIT, textos ticket, datos fiscales extendidos según migraciones)
Cientes FE (DIAN)	clients (documento, razón social, responsabilidades fiscales, dirección ampliada)
Proveedores	suppliers, supplier_payments
Caja / contabilidad	cash_sessions, cash_movements, payment_methods, transaction_categories, accounting_entries
Cuentas por cobrar/pagar	accounts_receivable, accounts_payable, receivable_payments, payable_payments

3.3 Integridad y copias

- Relaciones con FOREIGN KEY en tablas clave (p. ej. movimientos producto/usuario).
- Existen servicios de respaldo en el módulo de facturación electrónica (backup_service.dart).

4. Módulos y funcionalidades (detalle operativo)

4.1 Flujo de arranque y licenciamiento

1. Ruta inicial: /check-license ☒ comprueba activación vía LicenseService + SharedPreferences.
2. Si no está activada ☒ /activation (pantalla de activación / demo según implementación).
3. Si está activada ☒ /login.

4.2 Autenticación y autorización

- ☒ Login con usuario y contraseña contra SQLite.
- ☒ Contraseñas: soporte de almacenamiento hasheado (SecurityService); retrocompatibilidad con texto plano antiguo con migración automática a hash al iniciar sesión.
- ☒ Roles: admin, manager, supervisor, cashier, maintenance.
- ☒ Permisos granulares (Permission enum): productos, POS, anular ventas, reimpresión, inventario, reportes, configuración empresa/sistema, clientes, carrito, etc.
- ☒ Matriz por rol definida en RolePermissions; persistencia de ajustes en SharedPreferences.

4.3 Dashboard

- ☒ Panel lateral con acceso a módulos según permisos.
- ☒ Entradas típicas: Dashboard, Punto de Venta, Inventario, Movimientos, Clientes, Proveedores, Cuentas por cobrar, Reportes (contables), Configuración, Datos de empresa, Impresora POS, Usuarios, Permisos.

4.4 Punto de venta (POS)

- ☒ Selección de productos, carrito, descuentos, múltiples formas de pago.
- ☒ IVA por producto: 0 % (exento), 5 %, 19 % ☒ reflejado en líneas del carrito y totales.
- ☒ Productos pesables (campos de peso / precio por kg según modelo).
- ☒ Modo táctil y atajos de teclado (p. ej. reimpresión).
- ☒ Integración con impresión de recibo (PrintService) con desglose de IVA cuando aplica.
- ☒ Facturación electrónica desde el flujo de venta (pantallas del módulo FE).
- ☒ Cuentas por cobrar en ventas a crédito (servicio de cartera).

4.5 Inventario

- ☒ CRUD de productos, grupos, stock mínimo, imágenes.
- ☒ Formulario de producto con pestañas: información básica, producto exento de IVA (sincronizado con IVA guardado), facturación electrónica (IVA %, campos DIAN).
- ☒ Modos de precio: precio fijo / utilidad fija (margen).

4.6 Movimientos de inventario

- ☒ Registro de entradas, salidas, ajustes, consumo propio, etc.
- ☒ Búsqueda de producto con diálogo tipo ☒upa☒ (nombre/código) para evitar listas largas.

4.7 Clientes y proveedores

- ☒ Gestión de clientes del POS (puntos, historial).
- ☒ Proveedores y pagos a proveedores (tablas dedicadas).

4.8 Reimpresión de facturas (menú dedicado)

- ☒ Listado de ventas del día (y filtros).
- ☒ Reimprimir ticket.
- ☒ Anular venta (permiso cancelSales): marca anulada, devuelve stock, auditoría usuario/fecha.
- ☒ Registrar devolución: genera registro de devolución para reportes (distinto de anular).
- ☒ Filtro por estado: todas / activas / anuladas.

4.9 Reportes

- ☒ /reportes: reportes generales del sistema (pantalla ReportsScreen).
- ☒ /reportes-contables: AccountingReportsScreen ☒ cierre de caja, ventas por producto/categoría, devoluciones, IVA incluido, exportación PDF/Excel según implementación; líneas informativas de facturas canceladas y botón Actualizar para refrescar datos sin cerrar sesión de caja.

4.10 Cuentas por cobrar y por pagar

- ☒ Pantalla unificada AccountsReceivablePayableScreen.
- ☒ Modelos y servicios bajo modules/accounting/.

4.11 Configuración

- ☒ Datos de empresa: razón social, NIT, textos de ticket, datos de contacto y campos fiscales extendidos.
- ☒ Impresora POS: configuración para impresión (incl. serie).
- ☒ Usuarios y Permisos (solo roles con derecho de modificación).
- ☒ Códigos de autorización (pantalla/servicio dedicado para flujos que lo requieran).

4.12 Módulo de facturación electrónica (DIAN)

Rutas registradas:

Ruta	Pantalla
-----	-----
/facturacion-electronica	ElectronicInvoiceScreen (hub principal)
/electronic-invoicing/system-config	Configuración del sistema FE
/electronic-invoicing/status	Estado de facturas / documentos
/electronic-invoicing/queue	Cola de facturas pendientes de emisión o reproceso

Componentes típicos del módulo:

- ☒ Generación de documentos (document_generation_service.dart).
- ☒ Validación pre-emisión (pre_emission_validation_service.dart).
- ☒ Cola persistente (pending_invoice_queue_service.dart, modelos de cola).
- ☒ Estado y consulta (invoice_status_service.dart).
- ☒ Respaldo (backup_service.dart).
- ☒ Registro de auditoría (audit_log.dart).
- ☒ Widgets de UI: detalles, filtros, reintentos, validación visual.

Nota para junta: La facturación electrónica está acoplada funcionalmente al POS y catálogo (productos/clientes FE); la emisión real ante DIAN depende de la configuración cargada (certificados, ambiente, proveedor tecnológico) y del estado de los servicios externos ☒ aspecto a validar en despliegue con el área fiscal.

4.13 Otros

- ☒ /debug: pantalla de depuración (acceso restringido por uso interno).
- ☒ CheckLicenseScreen, ActivationScreen: ciclo de vida de licencia/demo.

5. Requisitos técnicos para desarrollo y ejecución

5.1 Desarrollo

Requisito	Notas
-----	-----
Flutter	Instalado y en PATH (Windows).
Dart	Incluido con Flutter (SDK acorde a pubspec).
Visual Studio (Workload C++)	Necesario para compilar Windows desktop.
Editor	Cursor / VS Code + extensión Dart/Flutter.

Requisito	Notas
Proyecto	Abrir carpeta que contiene pubspec.yaml (p. ej. raíz smart_seller según estructura del repo).

Comandos habituales:

```
flutter pub get
flutter run -d windows
```

5.2 Ejecución en producción (equipo cliente)

Requisito	Notas
-----	-----
SO	Windows 10/11 (64 bits) típico.
Runtime	Ejecutable generado por flutter build windows (no requiere Dart en el PC del cliente).
Almacenamiento	Espacio para BD SQLite y archivos de respaldo/exportación.
Periféricos	Impresora térmica compatible si se usa impresión por puerto serie (libserialport).
Red	Necesaria si se usa facturación electrónica contra servicios en línea o APIs externas.

5.3 Lo que el código no incluye por sí solo

- ☒ Servidor central de productos multi-sucursal en tiempo real (modelo offline-first local).
- ☒ Infraestructura cloud propia del fabricante en este repositorio (salvo integraciones configuradas en FE).
- ☒ Garantía legal de homologación DIAN sin validación en ambiente de pruebas/producción con certificado real.

6. Seguridad y cumplimiento (resumen honesto)

Tema	Estado en código
-----	-----
Autenticación	Usuario/contraseña local; hash con migración desde legado.
Autorización	Roles + permisos persistidos; middleware en rutas sensibles.
Datos en disco	SQLite local ☒ protección física del equipo y copias de seguridad son responsabilidad operativa.
Comunicaciones FE	Depende de TLS/configuración del stack de integración DIAN implementado en servicios del módulo.

7. Métricas aproximadas del código (referencia)

- ☒ ~115 archivos Dart bajo lib/ (proyecto DNPA).
- ☒ Dependencias directas listadas en pubspec.yaml (sección dependencies).

(Cifras orientativas; pueden variar con nuevos commits.)

8. Limitaciones y riesgos a declarar en junta

1. Dependencia de Flutter/Google para toolchain y actualizaciones del framework.
2. Datos locales: pérdida de disco sin backup implica pérdida de información comercial.
3. Facturación electrónica: requiere parametrización fiscal correcta y continuidad del proveedor/servicio ante la DIAN.
4. Un solo binario por instalación: escalabilidad multi-equipo sincronizado en tiempo real no es el modelo por defecto del diseño actual (cada instancia con su SQLite salvo integraciones adicionales no descritas aquí).

9. Documentos relacionados en el repositorio

- ☒ README.md (genérico Flutter en esta variante).
- ☒ README_COPIA_OBRA.txt (si aplica derechos de autor).

❏ keygen/README.md (herramientas de claves, si se usa en licenciamiento).

10. Contacto y mantenimiento

❏ Repositorio: según URL configurada en git remote (ej. <https://github.com/ingeniero82/POS.git>).

❏ Rama de trabajo reciente documentada en conversaciones de proyecto:
feature/facturacion-electronica-completa.

Documento generado a partir del análisis del código fuente del proyecto SmartSeller_POS_v1.0_DNPA. Cualquier cambio posterior en el repositorio puede alterar tablas, rutas o dependencias; se recomienda revisar pubspec.yaml, main.dart y sqlite_database_service.dart ante nuevas versiones.

Anexo: versión PDF

Existe una copia en PDF junto a este archivo:

❏ DOCUMENTACION_TECNICA_JUNTA_SMART_SELLER.pdf (misma carpeta docs/).

Para regenerar el PDF después de editar este Markdown, desde la raíz del proyecto DNPA ejecute:

```
dart run tool/build_junta_pdf.dart
```

(Requiere tener Dart/Flutter en el PATH y haber ejecutado antes flutter pub get en esa carpeta.)